Milestone 1 - Responsible User Module

Milestone 1 allows a responsible adult to sign up, log in, view their home screen (Dashboard), and create NNA users using dynamic forms.

1. **Responsible User Authentication**
    - **a.** - User Sign up
    - **b.** - User Login:
        - **i.** Secure authentication using Supabase Auth
        - **ii.** Session management (login / logout)
        - iii. Authenticate the user, determine their role, and redirect them to the correct view with preloaded permissions.
    - **c.** - User Password recovery
    - **d.** - Basic form validations and error handling

Image: Sign up IU

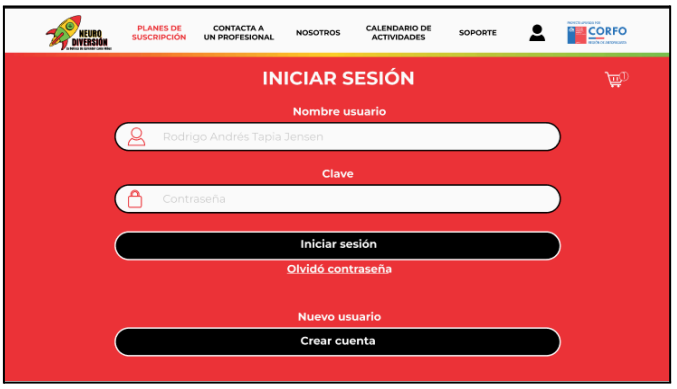


Image: Login IU

Images: Recovery password IU

After successfully logging in (active session), follow these steps to be redirected to the next page:

1. Obtaining the context: Invoking the backend RPC function ‘**get_app_context**’:
   const { data: context } = await supabase.rpc('get_app_context');

2. Response handling (Store & Redirect): The function returns a JSON object with two primary keys:

   a. ‘**permissions**(Persistence): Save the complete 'permissions' object in the application's Global State

      i. This object contains vital flags (view_type, primary_action, plan_status) that the next view will need to render. These variables do not need to be processed or read here; they are simply stored in memory so they are available throughout the session.

      ii. The object is polymorphic. Its content changes radically depending on the user's role.

         1. The ‘view_type’ variable will always be included. It is used to decide which layout to render.

         2. All other variables should be treated as optional (?) or nullable.

      iii. List of variables available in the JSON:

Variable	Type	Description
view_type	string	**Mandatory.** Define exactly which Dashboard to load: individual_client_dashboard, teacher_dashboard, admin_institution_dashboard or admin_neuro_dashboard**.**
primary_action	string	Suggested action for the main button (ADD_STUDENT or SUBSCRIBE).
slots_total	number	Total number of slots contracted in the current plan.
slots_used	number	Number of slots already used by own children.
has_institutional_nna	boolean	true if the user has students linked by an institution (read-only mode).
can_create_course	boolean	Permission to create new courses (Teacher Role).
can_invite_students	boolean	Permission to send invitations to students (Teacher Role).
can_manage_teachers	boolean	Permission to add or remove teachers (Institution Admin Role).
can_manage_billing	boolean	Permission to manage school payments and subscriptions (Institution Admin Role).
is_platform_admin	boolean	Superuser flag with total access (Neuro Admin Role).

   b. ‘**redirect_to**(Redirection): The system uses a Polymorphic Dashboard. There is a single destination path (/dashboard), but the content is rendered dynamically according to the user's role.

i. redirect_to: Use this for router.push()

ii. On the /dashboard page, read ‘permissions.view_type’ and load the corresponding component.

2. **Responsible_uto be dashboard. View_type name: “individual_client_dashboard”:**

This interface allows you to manage the subscription, view linked students in 2 different lists, and access the user menu.

a. **Data loading strategy** (Data Fetching):

i. Read the Context variables (stored in the global store during login) to configure the page skeleton.

Context Variable	Impact on the Interface
primary_action	Define what **Dynamic Button**(Section 1) is shown above.
slots_used / slots_total	Define the **Title with Accountant** from the student list
has_institutional_nna	Define whether the second list is rendered or hidden completely.

b. **Visual structure** (Layout)

i. **Main action button**This is the most important button on the screen. Its behavior changes dynamically.

OR

Main Button Logic (Based on ‘permissions.primary_action’ (`json`):

Value of primary_action	Button Text	Click Action
'ADD_STUDENT'	**"Agregar alumno(+)"**	Open Student Registration Form.
'SUBSCRIBE'	**"Ver planes de suscripción"**	Redirect to subscriptions page

ii. **Student lists:** Call the RPC function 'get_my_student_lists' which retrieves in a single call all students linked to the logged-in user, automatically separated into two lists:

1. **Input data:**

Parameter	Type	Description
-	-	**None.** The function automatically detects the user using their session token (auth.uid()).

2. **Example use:**

```
const { data, error } = await
supabase.rpc('get_my_student_lists');

const list1 = data.managed_students;
const list2 = data.monitored_students;
```

3. **Return data: JSON format:**

Field	Type	Description and frontend logic
managed_students	Array	My Children: List of students for whom the user is the Guardian. *(Enable Edit and Payment Management buttons)*.
monitored_students	Array	Institutional: List of students for whom the user is the Observer. *(Read-Only/Monitoring View)*.
↳ student_id	UUID	Unique identifier. Use as a key when rendering lists.
↳ full_name	String	Full name concatenated by the DB. Ready to display.

```
{
  // LIST 1:
  "managed_students": [
    {
      "student_id": "c0d3... (UUID)",
      "full_name": "Juanito Pérez"
    },
    {
      "student_id": "a1b2... (UUID)",
      "full_name": "Anita Pérez"
    }
  ],

  // LIST 2:
  "monitored_students": [
    {
      "student_id": "e5f6... (UUID)",
      "full_name": "Pedro Alumno"
    }
  ]
}
```

4. **List 1: “Alumnos registrados”:**This is the self-management section (children/tutors) with editing permissions.

   If the variable ‘slots_used’ == 0, the following should be displayed:

   Otherwise, display the list of students obtained in **list 1**

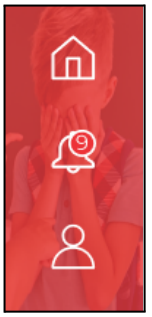
   - **Section Title** Format: "Registered Students ({slots_used}/{slots_total})"
   - Card name: ‘full_name’

5. **List 2: ‘Alumnos en seguimiento escolar’:**This is the institutional management section without editing permissions.

   If the variable ‘has_institutional_nna’ is FALSE, nothing should be displayed. Otherwise, the list 2 of students should be displayed

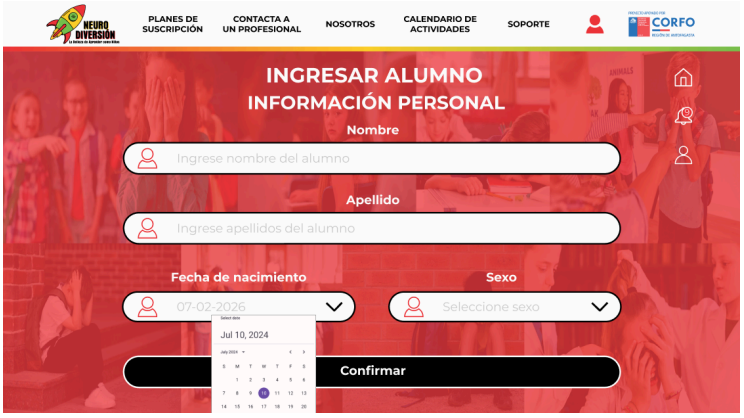
iii. Fixed side menu throughout the website

1. Home button: Returns to Responsible_user dashboard. View_type name: “individual_client_dashboard”
2. Notifications button (Future implementation)
3. User options button (Future implementation)



3. **Add ‘nna_user’ button:** This button displays the sequence of interfaces to enter a new 'nna_user'

   a. **Create student IU:** The first interface is static and is responsible for creating an 'nna_user' profile in the DB



After entering the data, the ‘Confirm’ button should call the rpc function: ‘**init_student_diagnosis’.** This function inserts the data and returns the redirect URL. Example usage:

   i. **Entry details:**

```
const { data, error } = await supabase.rpc('init_student_diagnosis', {
        p_first_name: 'Juan',
        p_last_name: 'Pérez',
        p_birthdate: '2018-05-20',
        p_sex: 'MALE'
        });
```

Parameter	JS Type	Format / Restriction
p_first_name	string	Plain text.
p_last_name	string	Plain text.
p_birthdate	date	ISO format'YYYY-MM-DD'.
p_sex	string	**CRITICAL:** enum type: ‘sex’

   ii. **Output data:**

Property	Type	Frontend Usage
redirect_url	string	**Main Action:** The backend returns a string: "/assessment/105/1". Each slash (/) after "assessment" represents entering a dynamic folder (variable).

		`app/`
		`└── assessment/`
		`  └── [instanceId]/  <-- Dynamic Folder 1 (Captures the 105)`
		`  └── [sectionOrder]/  <-- Dynamic Folder 2 (Captures the 1)`
		`  └── page.tsx  <-- The file that receives both data points`
		Use in`router.push(data.redirect_url)` immediately.

student_id	uuid	**Reference:** Save it if you need to update the local state or cache.
instance_id	bigint	**Back**: This is the ID of the created diagnosis.

Use: Disable the "Confirm" button (disabled={loading}) as soon as the user clicks and you activate a loading state (isLoading).

b. Diagnostic forms ('assessment' UI name):

The following interfaces load automatically after confirming the previous step (Only for certain subscription types will you be redirected to the dashboard view and skip the forms). The forms are loaded dynamically, using the rpc function: '**get_assessment_step**': Which loads all the information necessary to render a diagnostic screen: title, top navigation bar (stepper) and the questions (checklist).

i. Input data:

Parameter	Type	Origin	Description
p_instance_id	number	URL [instanceId]	**Unique ID of the diagnostic instance.**
p_section_order	number	URL [sectionOrder]	**Number of the section to display (1, 2, 3...).**

ii. Example of use:

```
const { data, error } = await supabase.rpc('get_assessment_step', {
        p_instance_id: 105,   // from params.instanceId
        p_section_order: 2    // from params.sectionOrder
});
```

iii. Output data:

Field	Type	Description / UI Logic
status	**String**	Flow Control. If it is 'COMPLETED' redirect immediately to /dashboard. If it is 'IN_PROGRESS', render.
current_section	**Int**	Validation. Must match the URL parameter[sectionOrder].
stepper	**Array**	Ready for the Top Progress Bar. Map this array.
↳ display_order	**Int**	Unique step ID. Use as key in the loop.
↳ label	**String**	Visible text of the step (e.g., "Memoria).
↳ state	**String**	Visual Style:'CURRENT'(bold white),'COMPLETED'(opacity and check icon),'LOCKED'(opacity).
↳ is_navigable	**Bool**	Interaction: If it is true enable on Click to navigate to that section. If it is false, disable pointer.
items	**Array**	List for the Central Form. Map this array.
↳ item_id	**Int**	Unique question ID. Use to save answers.
↳ question_text	**String**	Question text (Checkbox label).
↳ is_checked	**Bool**	Initial State: Default value for useState (Checked/Unchecked).

iv. Observations on implementation:

1. Make sure to include p_section_order in the useEffect dependency array.
2. **Initial State**Use item.is_checked to initialize the useState of your checkboxes.
3. **Blockade:**If the user tries to access a future blocked section via URL, the function will throw an error (capture it and redirect to the last valid section).
4. **Static texts in the interface:**

c. **Confirmation button:** Save the responses for the current section and return the navigation instruction. Call the rpc function 'submit_section_response'.

i. **Input data:**

Parameter	JS Type	Structure / Description
p_instance_id	Number	ID of the current instance (comes from the URL).
p_section_order	Number	Number of the section being saved.
p_responses	Array	Answer list. Format:[{ "item_id": 10, "value": true }].

ii. **Example of use:**

```
const saveAndContinue = async () => {
// 1. Prepare payload (example)
        const responsesPayload = [
      { item_id: 55, value: true },
        { item_id: 56, value: false }
  ];
// 2. Backend Call
  const { data, error } = await supabase.rpc('submit_section_response',
{
        p_instance_id : 105 , // Instance ID
      p_section_order: 2, // Current Section
        p_responses: responsesPayload
  });
```

iii. **Output data:**

Field	Type	Possible Values	Description / Frontend Logic
action	String	'NEXT_SECTION'	It indicates that the user is advancing to the next page of the test.
		'FINISH'	It indicates that the test has ended.
redirect_url	String	'/assessment/...'	If it's Next Section: Redirects to the next question.
		'/dashboard'	If it's Finish: Redirects to the base route. The Dashboard component will handle displaying the correct view (Teacher/Client).

iv. **Observation:** Block the "Save" button (isLoading) while the request is in flight to prevent duplicate submissions.